

ASCII table and history

Or, why does Ctrl+i insert a Tab in my terminal?

Dec	Hex	Binary	Char	Dec	Hex	Binary	Char	Dec	Hex	Binary	Char	Dec	Hex	Binary	Char
0	0x00	00 00000	NUL	32	0x20	01 00000	SPACE	64	0x40	10 00000	@	96	0x60	11 00000	`
1	0x01	00 00001	SOH	33	0x21	01 00001	!	65	0x41	10 00001	A	97	0x61	11 00001	a
2	0x02	00 00010	STX	34	0x22	01 00010	"	66	0x42	10 00010	B	98	0x62	11 00010	b
3	0x03	00 00011	ETX	35	0x23	01 00011	#	67	0x43	10 00011	C	99	0x63	11 00011	c
4	0x04	00 00100	EOT	36	0x24	01 00100	\$	68	0x44	10 00100	D	100	0x64	11 00100	d
5	0x05	00 00101	ENQ	37	0x25	01 00101	%	69	0x45	10 00101	E	101	0x65	11 00101	e
6	0x06	00 00110	ACK	38	0x26	01 00110	&	70	0x46	10 00110	F	102	0x66	11 00110	f
7	0x07	00 00111	BEL	39	0x27	01 00111	'	71	0x47	10 00111	G	103	0x67	11 00111	g
8	0x08	00 01000	BS	40	0x28	01 01000	(	72	0x48	10 01000	H	104	0x68	11 01000	h
9	0x09	00 01001	HT	41	0x29	01 01001	)	73	0x49	10 01001	I	105	0x69	11 01001	i
10	0x0a	00 01010	LF	42	0x2a	01 01010	*	74	0x4a	10 01010	J	106	0x6a	11 01010	j
11	0x0b	00 01011	VT	43	0x2b	01 01011	+	75	0x4b	10 01011	K	107	0x6b	11 01011	k
12	0x0c	00 01100	FF	44	0x2c	01 01100	,	76	0x4c	10 01100	L	108	0x6c	11 01100	l
13	0x0d	00 01101	CR	45	0x2d	01 01101	-	77	0x4d	10 01101	M	109	0x6d	11 01101	m
14	0x0e	00 01110	SO	46	0x2e	01 01110	.	78	0x4e	10 01110	N	110	0x6e	11 01110	n
15	0x0f	00 01111	SI	47	0x2f	01 01111	/	79	0x4f	10 01111	O	111	0x6f	11 01111	o
16	0x10	00 10000	DLE	48	0x30	01 10000	0	80	0x50	10 10000	P	112	0x70	11 10000	p
17	0x11	00 10001	DC1	49	0x31	01 10001	1	81	0x51	10 10001	Q	113	0x71	11 10001	q
18	0x12	00 10010	DC2	50	0x32	01 10010	2	82	0x52	10 10010	R	114	0x72	11 10010	r
19	0x13	00 10011	DC3	51	0x33	01 10011	3	83	0x53	10 10011	S	115	0x73	11 10011	s
20	0x14	00 10100	DC4	52	0x34	01 10100	4	84	0x54	10 10100	T	116	0x74	11 10100	t
21	0x15	00 10101	NAK	53	0x35	01 10101	5	85	0x55	10 10101	U	117	0x75	11 10101	u
22	0x16	00 10110	SYN	54	0x36	01 10110	6	86	0x56	10 10110	V	118	0x76	11 10110	v
23	0x17	00 10111	ETB	55	0x37	01 10111	7	87	0x57	10 10111	W	119	0x77	11 10111	w
24	0x18	00 11000	CAN	56	0x38	01 11000	8	88	0x58	10 11000	X	120	0x78	11 11000	x
25	0x19	00 11001	EM	57	0x39	01 11001	9	89	0x59	10 11001	Y	121	0x79	11 11001	y
26	0x1a	00 11010	SUB	58	0x3a	01 11010	:	90	0x5a	10 11010	Z	122	0x7a	11 11010	z
27	0x1b	00 11011	ESC	59	0x3b	01 11011	;	91	0x5b	10 11011	[	123	0x7b	11 11011	{
28	0x1c	00 11100	FS	60	0x3c	01 11100	<	92	0x5c	10 11100	\	124	0x7c	11 11100	
29	0x1d	00 11101	GS	61	0x3d	01 11101	=	93	0x5d	10 11101	]	125	0x7d	11 11101	}
30	0x1e	00 11110	RS	62	0x3e	01 11110	>	94	0x5e	10 11110	^	126	0x7e	11 11110	~
31	0x1f	00 11111	US	63	0x3f	01 11111	?	95	0x5f	10 11111	_	127	0x7f	11 11111	DEL

The binary representation has the most significant bit first (“big endian”).

ASCII is 7-bit; because many have called encodings such as [CP437](#), [ISO-8859-1](#), [CP-1252](#), and others “extended ASCII” some are under the misapprehension that ASCII is 8-bit (1 byte).

Understanding ASCII (and terminals)

To understand why Control+i inserts a Tab in your terminal you need to understand ASCII, and to understand ASCII you need know a bit about its history and the world it was developed in. Please bear with me.

Teleprinters

Teleprinters evolved from the telegraph. Connect a printer and keyboard to a telegraph and you’ve got a teleprinter. Early versions were called “printing telegraphs”.

Most teleprinters communicated using the ITA2 protocol. For the most part this would just encode the alphabet, but there are a few control codes: WRU (“Who R U”) would cause the receiving teleprinter to send back its identification, BEL would ring a bell, and it had the familiar CR (Carriage Return) and LF (Line Feed).



Image 1, a printing telegraph produced in 1907. The alphabetically sorted piano keys are a great example of how the first generation of new innovations tends to resemble whatever already exists, and that it takes a few more innovations to

This is all early 20th century stuff. There are no electronic computers; it’s all mechanical working with punched tape. ITA2 (and codes like it) were mechanically efficient; common letters such as “e” and “t” required only a single hole to be punched.

These 5-bit codes could only encode 32 characters, which is not even enough for just English. The solution was to add the FIGS and LTRS codes, which would switch between “figures” and “letters” mode. “FIGS R W” would produce “42”. This worked, but typo’ing a FIGS or LTRS (or losing one in line noise) would result in gibberish. Not ideal.

Terminals

In the 1950s teleprinters started to get connected to computers, rather than other teleprinters. ITA2 was designed for mechanical machines and was awkward to use. ASCII was designed specifically for computer use and published in 1962. Teleprinters used with computers were called *terminals* (as in “end of a connection”, like “train terminal”). Teleprinters were also called “TeleTYpewriter”, or TTY for short, and you can still find names like /dev/tty or /bin/stty on modern systems.

People really programmed computers using teleprinters. Here’s a [video of a teleprinter in action](#), and here’s a somewhat cheesy (but interesting and cute) video which explains how they [were used to program a PDP 11/10](#).

A terminal would connect to a computer with a serial port ([RS-232](#)), which simply transfers bytes back and forth. A terminal is more akin to a monitor with a keyboard, rather than a computer on its own. A modern monitor connected with HDMI is told “draw this pixel in this colour”, in the 1960s the computer merely said “here are a bunch of characters”.

If you’re wondering what a “shell” is: a shell is a program to interact with your computer. It provides a commandline, runs programs, and displays the result. The terminal just displays characters. It’s the difference between a TV and a DVD player.

Teleprinters needed some way to communicate events such as “stop sending me data” or “end of transmission”. This is what control characters are for. The exact meaning of control characters has varied greatly over the years (which is why extensive [termcap databases](#) are required). ASCII is more than just a character set; it’s a way to communicate between a terminal and a computer.

An additional method to communicate are [escape sequences](#). This is a list of characters starting with the ESC control character (0x1b). For example F1 is <Esc>OP and the left arrow is <Esc>[OD. Computers can give instructions to terminals, too: <Esc>[2C is move the cursor 2 positions forward and <Esc>[4m underlines all subsequent text. This is also how the Alt key works: Alt+a is <Esc>a.

Modern systems and ASCII properties

All of this matters because modern terminals operate on the same principles as those of the 1960s. If you’re opening three xterm or iTerm2 windows then you’re emulating three terminals connecting to a “mainframe”.

If you look at the ASCII table above then there are some interesting properties: in the 1st column you can see how the left two bits are always set to zero, and that the other 5 bits count to 31 (32 characters in total; it starts at 0). The 2nd column repeats this pattern but with the 6th bit set to 1 (remember, read binary numbers from right-to-left, so that’s 6th counting from the right). The 3rd column repeats this pattern again with the 7th bit set, and the final column has both bits set.

The interesting part here is that the letters A-Z and some punctuation map directly to the control characters in the 1st column. All that’s needed is removing one bit, and that’s exactly what the Control key did: clear the 7th bit. Lowercase and uppercase letters align in the 3rd and 4th columns, and this is what the Shift key did: clear the 6th bit.

Pressing Control+i (lowercase) would mean sending “i)”, which is not very useful. So most terminals interpret this as Control+I (uppercase), which sends HT. DEL is last is so all bits are set to 1. This is how you “deleted” a character in punch tapes: punch all the holes!

This is kind of neat and well designed, but for us it means:

- There is no way to see if the user pressed only Control or Shift, because from a terminal’s perspective all they do is modify a bit for the typed character.
- There is no way to distinguish between the Tab key and Control+i. It’s not just ‘the same’ as Tab, Control+i *is* Tab.

really get the most out of it. This style of piano keyboards was introduced in the 1840s, and while the keyboard as we know it today was introduced in the 1870s, it took a while for it to replace all piano-style keyboards; this is probably among the last models that was made).



Image 2, the Teletype model 33 ASR, introduced in 1963. This is one first ASCII teleprinters. Note the machinery on the left; you could feed this with a punched tape to automatically type a program for you, similar to how you would now load a program from a disk. The Teletype model 33 was massively popular, and the brand name Teletype became synonymous with terminal.



Image 3, Ken Thompson working on the PDP-11 using a Teletype (model 33?). What always struck be about this image is the atrocious ergonomics of ... everything. The keyboard, the chair, everything about the posture: it’s all terrible. Inventing Unix almost seems easy compared to dealing with that!



- There is no way to distinguish between Control+a and Control+Shift+a.
- Sending Control with a character from the 2nd column is useless. Control clears the 7th bit, but this is already 0, so Control+# will just send “#”.

The world has not completely stood still and there have been improvements since the 1960s, but terminals are still fundamentally ASCII-based text interfaces, and programs running inside a terminal – like a shell or Vim – still have very limited facilities for modern key events. Non-terminal programs don’t have these problems as they’re not restricted to a 1960s text interface.

Note: for brevity’s sake many aspects have been omitted in the above: ITA2 was derived from Murray code, the 1967 ASCII spec changed many aspects (1962 ASCII only had uppercase), there were other encodings (e.g. EBCDIC), graphical terminals such as the Tektronix 4014 (which xterm can emulate), ioctls, etc. References and further reading: [An annotated history of some character codes](#), [7-bit character sets](#), [Control characters in ASCII and Unicode](#), [The TTY demystified](#)

Image 4, DEC VT100, a kind of terminal that a terminal emulator such as xterm emulates. It has a visual display and supports the essential escape sequences still in use today. These were known as “visual terminals”, referring to the visual screen with characters, as opposed to printing them out.

Created by [Martin Tournoij](#), because I’ve had to explain “Control+i *is* Tab” once too many times and figured an in-depth explanation would be helpful.

[Source on GitHub](#); PRs and issues welcome.

Image credits: [Image 1 by Science Museum](#); [CC BY-NC-SA](#) | [Image 2 by AlisonW](#); [CC BY-SA](#) | [Image 3 by Peter Hamer](#); [CC BY-SA](#) | [Image 4 by Jason Scott](#); [CC BY](#)